

CeTZ Gallery

Ten diagrams in idiomatic CeTZ 0.5.2, each chosen for a technique worth copying.

[View as PDF](#). [View as HTML](#). [View source](#).

Every figure is a CeTZ redrawing of a TikZ figure from Walmes M. Zeviani's *Tikz Gallery* [1]. Each entry links its original. The originals remain his work and are not copied into this project. The redrawings translate the labels from Portuguese to English, use Typst's built-in colors, and differ in detail.

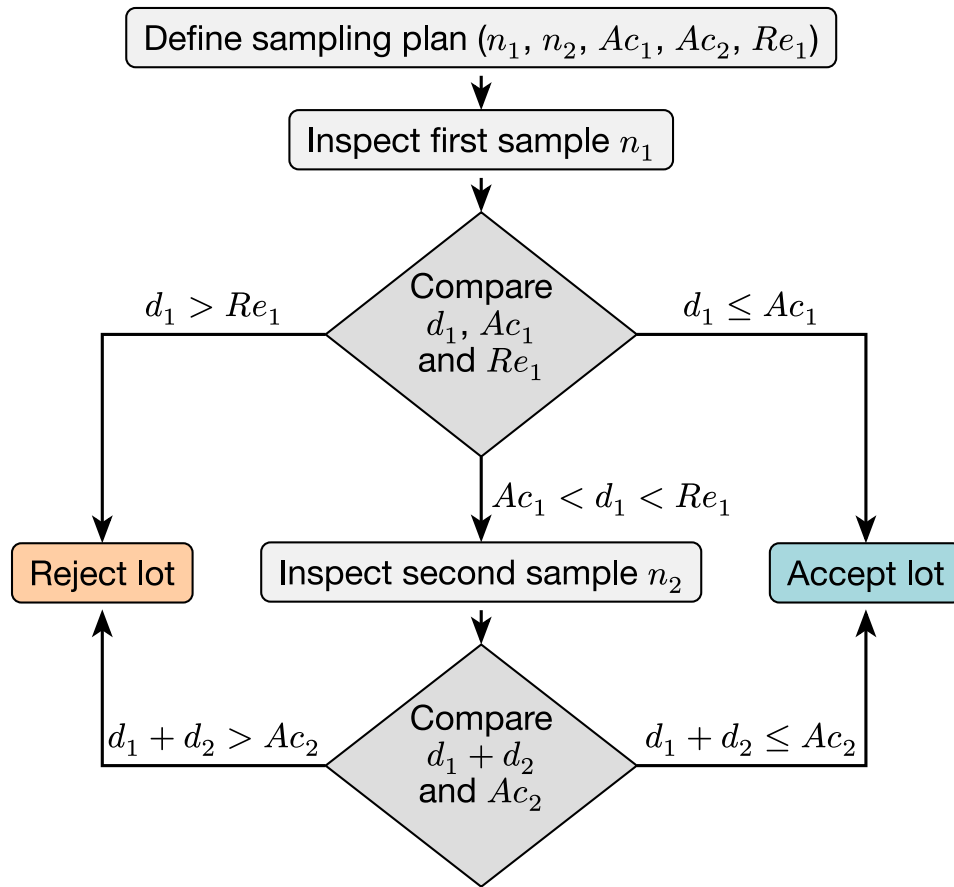
[1] Walmes M. Zeviani, *Tikz Gallery*. <https://github.com/walmes/Tikz>, <http://leg.ufpr.br/~walmes/Tikz/>.

Figures

Sampling-plan flowchart	2
Series-parallel blocks	4
System with inputs and output	5
Overlapping groups	6
Probability tree	8
Cycle on a triangle	9
Density with a shaded tail	11
Factorial-design cube	13
Interval with braces	15
Directory tree	16

Sampling-plan flowchart

Redrawn from [sampling-plan-2.pg](#) in Walmes M. Zeviani's *Tikz Gallery*. Relative placement from anchors; diamond decisions built from a text box's anchors; elbow branches with labels; two arrows meeting one box.



```

#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// Double-sampling acceptance plan: Ac = acceptance number, Re = rejection number.
#let Ac = $A c$
#let Re = $R e$

#cezt.canvas({
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)

  // A Typst box gives rounded corners while the anchors stay on its bounding box.
  let step(pos, name, body, fill: silver.lighten(60%), anchor: "north") = content(pos, name:
name,
  anchor: anchor, box(body, fill: fill, stroke: 0.6pt + black, radius: 3pt, inset: 5pt))
  // A diamond whose edge midpoints touch the corners of the text box.
  let decision(pos, name, body) = group(name: name, anchor: "north", {
    content(pos, box(width: 4.5em, align(center, par(leading: 0.35em, body))), name: "text")
    on-layer(-1, line(..("north", "east", "south", "west").map(a => ("text", 200%, "text." +
a)),
      close: true, fill: gray.lighten(60%)))
  })
  let arrow = line.with(stroke: 0.9pt + black, mark: (end: "stealth", fill: black))
  let branch(from, to, label) = {

```

```

    let corner = (from, "-|", to)
    arrow(from, corner, to)
    content((from, 50%, corner), label, anchor: "south", padding: 0.1)
  }

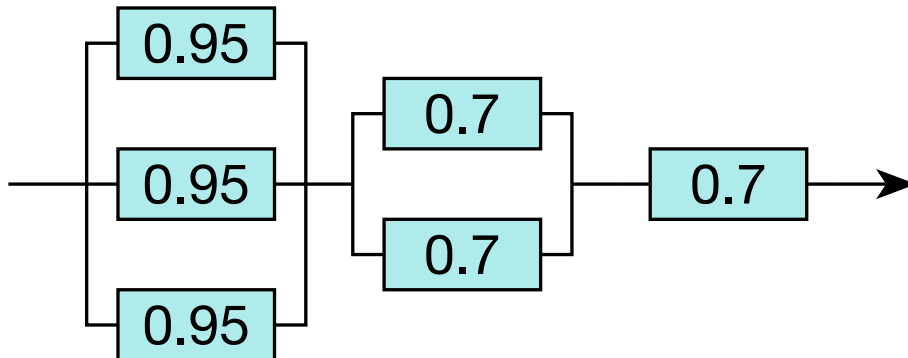
  let gap = 0.35
  step((0, 0), "plan", [Define sampling plan ($n_1$, $n_2$, $Ac_1$, $Ac_2$, $Re_1$)])
  step((rel: (0, -gap), to: "plan.south"), "ins1", [Inspect first sample $n_1$])
  decision((rel: (0, -gap), to: "ins1.south"), "d1", [Compare $d_1$, $Ac_1$ and $Re_1$])
  step((rel: (0, -2.5 * gap), to: "d1.south"), "ins2", [Inspect second sample $n_2$])
  decision((rel: (0, -gap), to: "ins2.south"), "d2", [Compare $d_1 + d_2$ and $Ac_2$])
  step((rel: (2 * gap, 0), to: "ins2.east"), "accept", [Accept lot],
    fill: eastern.lighten(60%), anchor: "west")
  step((rel: (-2 * gap, 0), to: "ins2.west"), "reject", [Reject lot],
    fill: orange.lighten(60%), anchor: "east")

  arrow("plan", "ins1")
  arrow("ins1", "d1")
  arrow("d1", "ins2")
  content(("d1.south", 50%, "ins2.north"), $Ac_1 < d_1 < Re_1$, anchor: "west", padding: 0.1)
  arrow("ins2", "d2")
  branch("d1.east", "accept.north", $d_1 \leq Ac_1$)
  branch("d1.west", "reject.north", $d_1 > Re_1$)
  branch("d2.east", "accept.south", $d_1 + d_2 \leq Ac_2$)
  branch("d2.west", "reject.south", $d_1 + d_2 > Ac_2$)
})

```

Series-parallel blocks

Redrawn from [circuitos_mistos1.pgf](#) in Walmes M. Zeviani's *Tikz Gallery*. A data-driven loop; forks and joins with perpendicular coordinates; wires drawn behind the boxes with `on=layer`.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

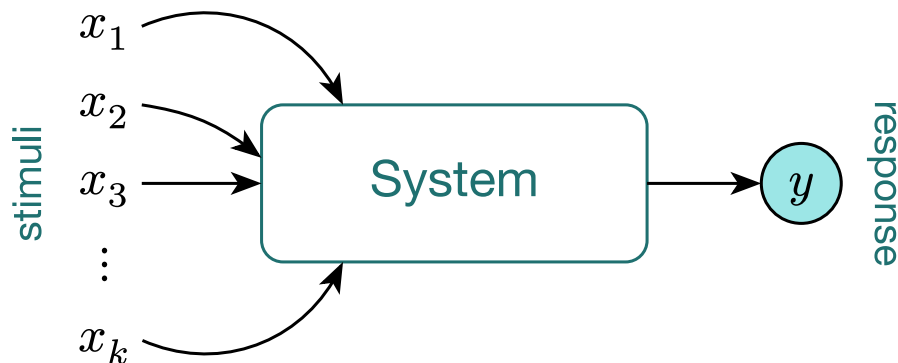
// Series-parallel reliability block diagram: each stage holds redundant
// components in parallel, and the stages are connected in series.
#cezt.canvas({
  import cetz.draw: *
  set-style(stroke: 0.6pt + black)

  let stages = ((0.95, 3), (0.7, 2), (0.7, 1)) // (reliability, count)
  let (dx, dy, w, h) = (1.7, 0.9, 1, 0.45)

  let prev = (rel: (-0.5, 0), to: "s0.west") // resolved when drawn, after "s0" exists
  for (i, (p, n)) in stages.enumerate() {
    let s = "s" + str(i)
    // The x-padding puts the group's west/east anchors on the fork and join buses.
    group(name: s, padding: (x: 0.2), {
      for k in range(n) {
        rect((i * dx, ((n - 1) / 2 - k) * dy), (rel: (w, h)), anchor: "center",
          fill: teal.lighten(60%), name: str(k))
        content(str(k), [#p])
      }
    })
    // Wires pass straight through the boxes, so draw them behind.
    on-layer(-1, {
      let (fork, join) = (s + ".west", s + ".east")
      line(prev, fork)
      for k in range(n) {
        let b = s + "." + str(k)
        line(fork, (fork, "|-", b), (join, "|-", b), join)
      }
    })
    prev = s + ".east"
  }
  line(prev, (rel: (0.5, 0)), mark: (end: "stealth", fill: black))
})
```

System with inputs and output

Redrawn from [reg_sistema.pgf](#) in Walmes M. Zeviani's *Tikz Gallery*. Curved connectors ending at border anchors given as angles; a circle joined to a box; rotated captions.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#let accent = teal.darken(45%)

#cetx.canvas({
  import cetx.draw: *
  set-style(
    stroke: 0.6pt + black,
    mark: (end: "stealth", fill: black, scale: 0.8),
    content: (padding: 0.08),
  )

  // Cubic curve leaving `a` and entering `b` at `angle` to the chord, like TikZ `bend left`.
  let bend(a, b, angle) = bezier(a, b, (a, 40%, angle, b), (b, 40%, -angle, a))

  rect((0, 0), (rel: (2.7, 1.1)), radius: 0.15, stroke: accent, name: "sys")
  content("sys", text(accent)[System])

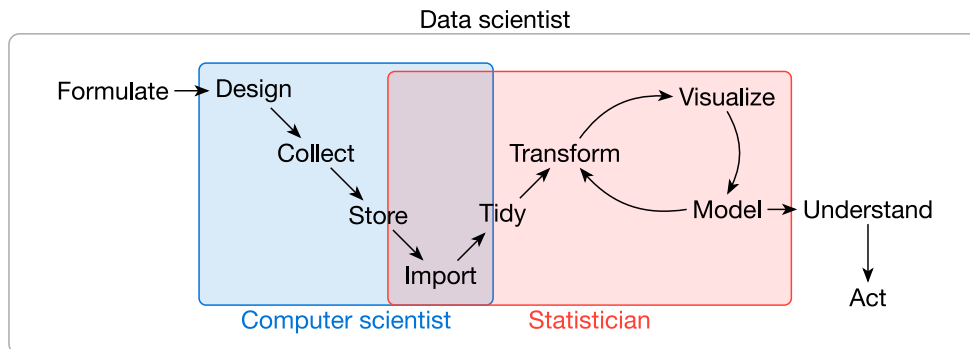
  let inputs = (x1: $x_1$, x2: $x_2$, x3: $x_3$, dots: $dots.v$, xk: $x_k$)
  for (i, (name, x)) in inputs.pairs().enumerate() {
    content((rel: (-1.1, (2 - i) * 0.55), to: "sys.west"), x, name: name)
  }
  bend("x1.east", (name: "sys", anchor: 120deg), 45deg)
  bend("x2.east", (name: "sys", anchor: 162deg), 15deg)
  line("x3", "sys")
  bend("xk.east", (name: "sys", anchor: 240deg), -45deg)

  circle((rel: (0.8, 0), to: "sys.east"), radius: 0.28, anchor: "west",
    fill: teal.lighten(50%), name: "y")
  content("y", $y$)
  line("sys", "y")

  set-style(content: (wrap: text.with(8pt, accent)))
  content((rel: (-0.1, 0), to: "x3.west"), [stimuli], angle: 90deg, anchor: "south")
  content((rel: (0.1, 0), to: "y.east"), [response], angle: -90deg, anchor: "south")
})
```

Overlapping groups

Redrawn from [data-science-workflow.pgf](#) in Walmes M. Zeviani's *Tikz Gallery*. Translucent frames around groups of nodes with `rect-around`; a cycle of bent arrows; placement by polar offsets.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetx.canvas({
  import cetx.draw: *
  set-style(
    stroke: 0.6pt + black,
    mark: (end: "stealth", fill: black, scale: 0.8),
    content: (padding: 0.08),
  )

  // Bare element names make `line` stop at both borders.
  let step(from, name, offset) = {
    content((rel: offset, to: from), name, name: name)
    line(from, name)
  }

  // Cubic curve leaving `a` and entering `b` at `angle` to the chord, like TikZ `bend left`.
  let bend(a, b, angle) = bezier(a, b, (a, 40%, angle, b), (b, 40%, -angle, a))

  content((0, 0), "Formulate", name: "Formulate")
  let prev = "Formulate"
  for (name, offset) in (
    ("Design", (0deg, 2)), ("Collect", (-45deg, 1.25)), ("Store", (-45deg, 1.25)),
    ("Import", (-45deg, 1.25)), ("Tidy", (45deg, 1.25)), ("Transform", (45deg, 1.25)),
  ) {
    step(prev, name, offset)
    prev = name
  }

  content((rel: (2.3, 0.8), to: "Transform"), "Visualize", name: "Visualize")
  content((rel: (2.3, -0.8), to: "Transform"), "Model", name: "Model")
  bend((name: "Transform", anchor: 45deg), "Visualize.west", 30deg)
  bend("Visualize.south", "Model.north", 30deg)
  bend("Model.west", (name: "Transform", anchor: -45deg), 30deg)

  step("Model", "Understand", (0deg, 2))
  step("Understand", "Act", (-90deg, 1.25))

  on-layer(-1, {
    for (members, hue, caption) in (
      (("Design", "Import"), blue, [Computer scientist]),
      (("Import", "Visualize"), red, [Statistician]),
    ) {
      rect-around(., members, padding: 0.15, radius: 0.1, name: "group",
```

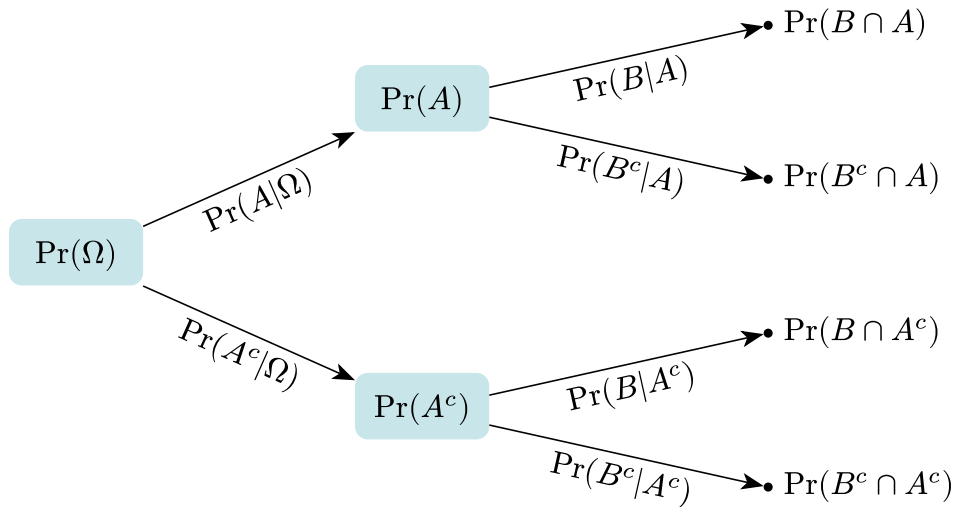
```

        stroke: hue, fill: hue.transparentize(85%)
    content("group.south", text(hue, caption), anchor: "north")
}
rect-around("Formulate", "Import", "Understand", "Act", padding: 0.6, radius: 0.15,
    stroke: gray, name: "all")
content("all.north", [Data scientist], anchor: "south")
})
})

```

Probability tree

Redrawn from [probtreescheme.pgf](#) in Walmes M. Zeviani's *Tikz Gallery*. `cetz.tree` growing to the right; custom node and edge callbacks; labels along the edges.



```

#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetz.canvas({
  import cetz.draw: *
  set-style(
    stroke: .5pt + black,
    mark: (end: "stealth", fill: black),
    content: (padding: .12),
  )

  let draw-node(n) = if n.depth < 2 {
    let bag = box.with(fill: eastern.lighten(75%), radius: 4pt, inset: 7pt, width: 4.2em)
    content((0, 0), padding: 0, bag(align(center, $Pr(#n.content)$)))
  } else {
    circle((0, 0), radius: .05, fill: black, stroke: none)
  }

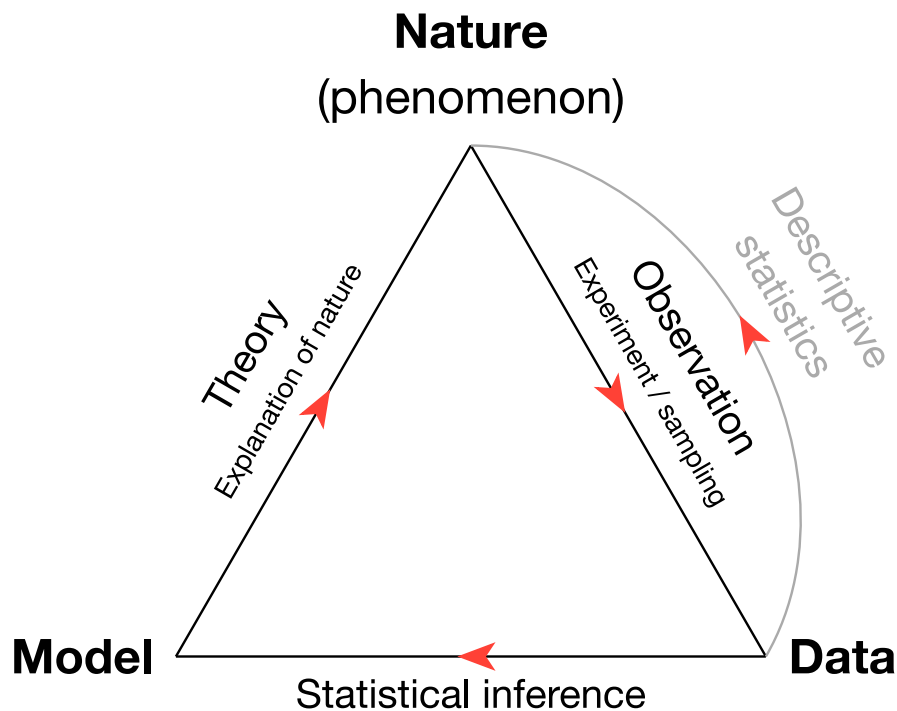
  let draw-edge(parent, child) = {
    let (a, b) = (parent.content, child.content)
    line(parent.group-name, child.group-name, name: "e")
    content("e.mid", $Pr(#b|#a)$, angle: "e.end", anchor: "north")
    // Only the edge callback sees both events, so it also labels the leaf.
    if child.depth == 2 {
      content(child.group-name + ".east", $Pr(#b inter #a)$, anchor: "west")
    }
  }

  // With direction "right", children are listed bottom to top.
  cetz.tree.tree(
    ($0omega$, ($A^c$, $B^c$, $B$), ($A$, $B^c$, $B$)),
    direction: "right", grow: 3, spread: 1.6,
    draw-node: draw-node, draw-edge: draw-edge,
  )
})

```

Cycle on a triangle

Redrawn from [knowledge-cycle.pgf](#) in Walmes M. Zeviani's *Tikz Gallery*. Polar coordinates; arrowheads in the middle of each edge; text rotated along edges; a Bézier curve with polar control points.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#let note(title, sub) = align(center)[#title \ #text(size: 7pt, sub)]

#cetz.canvas({
  import cetz.draw: *
  set-style(
    stroke: .6pt + black,
    // Mid-path arrowhead; shorten-to: none keeps CetZ from cutting the path at the mark.
    mark: (end: "stealth", pos: 50%, anchor: "center", shorten-to: none,
      scale: 1.4, fill: red, stroke: red),
    content: (padding: .2),
  )

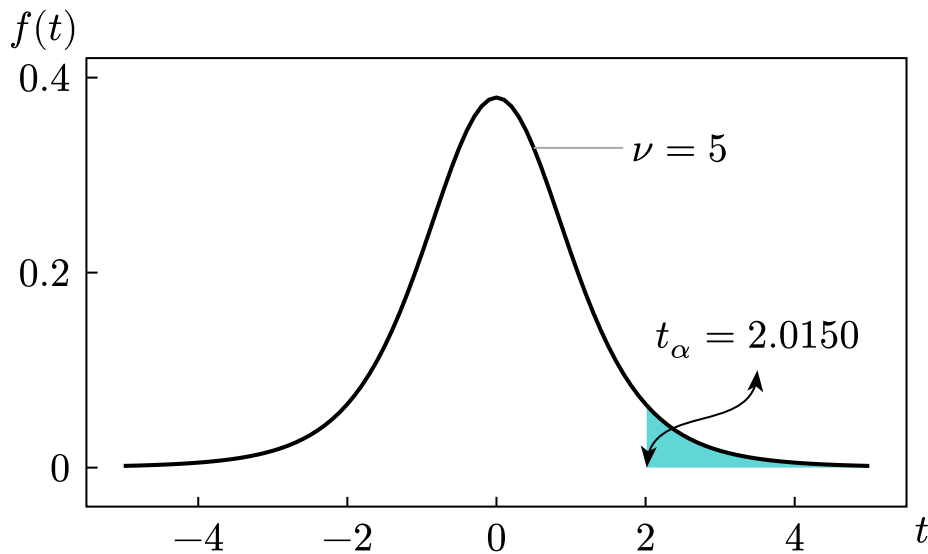
  anchor("nature", (90deg, 3))
  anchor("model", (210deg, 3))
  anchor("data", (330deg, 3))
  content("nature", text(12pt, align(center)[*Nature* \ (phenomenon)]), anchor: "south")
  content("model", text(12pt)[*Model*], anchor: "east")
  content("data", text(12pt)[*Data*], anchor: "west")

  let edge(a, b, label) = {
    line(a, b, name: "e")
    content("e.mid", label, angle: "e.end", anchor: "south")
  }
  edge("model", "nature", note[Theory][Explanation of nature])
  edge("nature", "data", note[Observation][Experiment / sampling])
  line("data", "model", name: "inference")
  content("inference.mid", [Statistical inference], anchor: "north")
})
```

```
bezier("data", "nature", (rel: (60deg, 2), to: "data"), (rel: (0deg, 2), to: "nature"),  
  name: "arc", stroke: gray, mark: (fill: red, stroke: red))  
content("arc.mid", angle: -60deg, anchor: "south",  
  text(fill: gray, align(center)[Descriptive \ statistics]))  
})
```

Density with a shaded tail

Redrawn from [dist_t_quantil.pgf](#) in Walms M. Zeviani's *Tikz Gallery*. A plot in data units via `scale`; a sampled curve; a filled tail; tick loops; a curved callout.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

#cetz.canvas({
  import cetz.draw: *
  set-style(stroke: (paint: black, thickness: .5pt), mark: (fill: black, scale: .7))
  // Data coordinates (t, f(t)); text, strokes and marks keep their size.
  scale(x: .65, y: 8.5)

  // Student t density for ν = 5: Γ(3) / (√(5π) Γ(5/2)) · (1 + t²/5)^(-3).
  let pdf(t) = 8 / (3 * calc.pi * calc.sqrt(5)) * calc.pow(1 + t * t / 5, -3)
  let curve(a, b) = range(101).map(i => a + (b - a) * i / 100).map(t => (t, pdf(t)))
  let q = 2.015

  line((q, 0), ..curve(q, 5), (5, 0), close: true, stroke: none,
    fill: teal.lighten(20%))
  line(..curve(-5, 5), stroke: (thickness: 1pt))

  rect((-5.5, -.04), (5.5, .42), name: "frame")
  for x in (-4, -2, 0, 2, 4) {
    let p = ((x, 0), "|-", "frame.south")
    line(p, (rel: (0, .01)))
    content(p, $#x$, anchor: "north", padding: 4pt)
  }
  for y in (0, .2, .4) {
    let p = ("frame.west", "|-", (0, y))
    line(p, (rel: (.15, 0)))
    content(p, $#y$, anchor: "east", padding: 4pt)
  }
  content("frame.south-east", $t$, anchor: "north-west", padding: 2pt)
  content("frame.north-west", $f(t)$, anchor: "south-east", padding: 2pt)

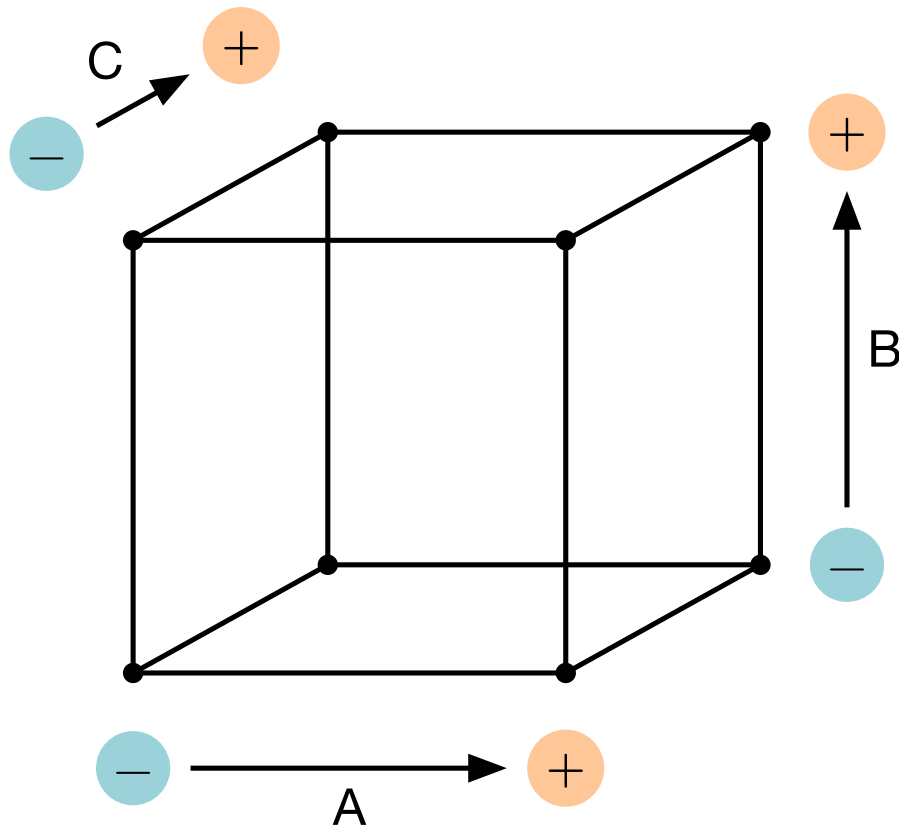
  line(.5, pdf(.5), (rel: (1.2, 0)), stroke: gray, name: "leader")
  content("leader.end", $nu = 5$, anchor: "west", padding: 2pt)

  bezier((q, 0), (3.5, .1), (q, .07), (3.5, .03),
```

```
mark: (start: "stealth", end: "stealth"), name: "callout")
content("callout.end", $t_alpha = 2.0150$, anchor: "south", padding: 3pt)
})
```

Factorial-design cube

Redrawn from [cubos-fatoriais-2a3.pg](#) in Walmes M. Zeviani's *Tikz Gallery*. An oblique 3D view from the canvas's z vector; a loop over vertices; framed circle markers joined by arrows.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// 2^3 factorial design: each vertex is one run, each axis one factor at levels -/+.
// Oblique projection: the canvas draws the z unit vector as (.45, .25).
#cetz.canvas(length: 3cm, z: (.45, .25), {
  import cetz.draw: *
  set-style(stroke: black, mark: (fill: black))

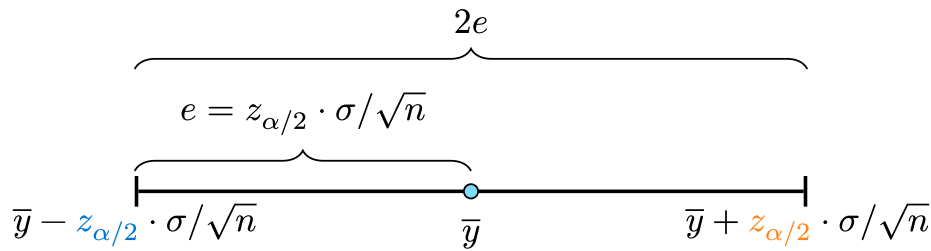
  for x in (0, 1) { for y in (0, 1) { for z in (0, 1) {
    let p = (x, y, z)
    if x == 0 { line(p, (rel: (1, 0, 0))) }
    if y == 0 { line(p, (rel: (0, 1, 0))) }
    if z == 0 { line(p, (rel: (0, 0, 1))) }
    circle(p, radius: 2pt, fill: black, stroke: none)
  } } }

  // Level discs beside the low and high vertex; a line between named
  // elements stops at their borders.
  set-style(content: (frame: "circle", stroke: none, padding: 1.5pt))
  let factor(id, low, high, offset, anchor) = {
    content((rel: offset, to: low), $-$, fill: eastern.transparentize(55%), name: id + "-")
    content((rel: offset, to: high), $+$, fill: orange.transparentize(55%), name: id + "+")
    // The invisible start mark makes `offset` leave a gap at both ends.
    line(id + "-", id + "+", mark: (start: (symbol: "|", stroke: none), end: ">", offset: 4pt))
    content((id + "-", 50%, id + "+"), id, anchor: anchor, frame: none, padding: 4pt)
```

```
}  
factor("A", (0, 0, 0), (1, 0, 0), (0, -.22), "north")  
factor("B", (1, 0, 1), (1, 1, 1), (.2, 0), "west")  
factor("C", (0, 1, 0), (0, 1, 1), (-.2, .2), "south-east")  
})
```

Interval with braces

Redrawn from [confidence-interval-mean-components.pgfi](#) in Walmes M. Zeviani's *Tikz Gallery*. flat-brace with labels; interval end marks; colored parts of math labels.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// Margin of error; `fill` colors the z.
#let half(fill) = $#text(fill: fill, $z_{(\alpha\backslash2)}$) dot sigma \ / sqrt(n)$

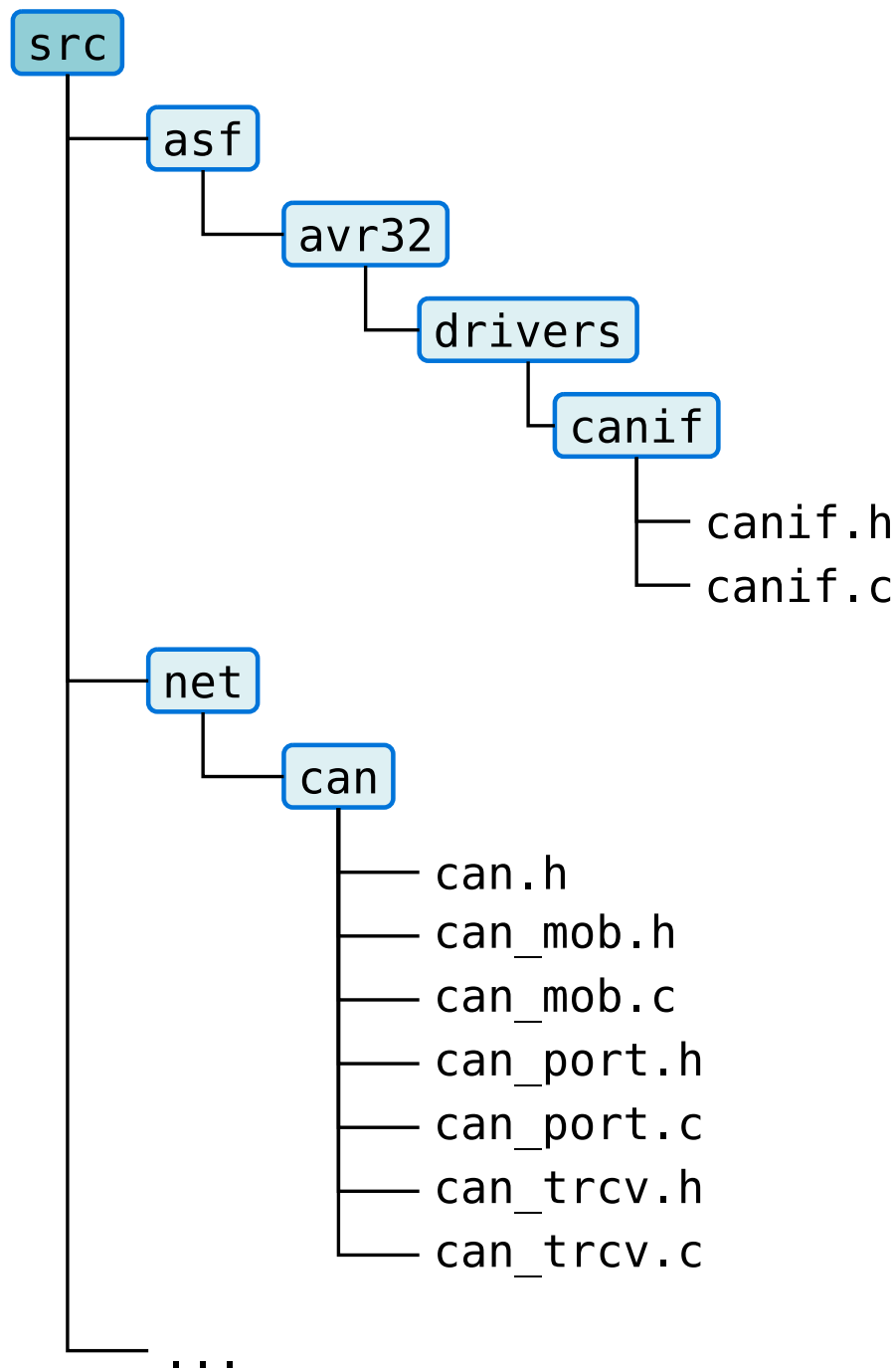
#set canvas({
  import cetz.draw: *
  import cetz.decorations: flat-brace
  set-style(
    stroke: (paint: black, thickness: .5pt),
    content: (padding: .15),
    flat-brace: (amplitude: .2, content-offset: 0),
  )

  line((-3.3, 0), (3.3, 0), name: "ci", stroke: (thickness: 1pt), mark: (symbol: "|",
width: .3))
  circle("ci.50%", radius: .07, fill: aqua)
  content("ci.50%", $\overline{y}$, anchor: "north", padding: .25)
  content("ci.start", $\overline{y} - \text{half(blue)}$, anchor: "north")
  content("ci.end", $\overline{y} + \text{half(orange)}$, anchor: "north")

  flat-brace((rel: (0, .2), to: "ci.start"), (rel: (0, .2), to: "ci.50%"), name: "e")
  content("e.content", $e = \text{half(black)}$, anchor: "south")
  flat-brace((rel: (0, 1.2), to: "ci.start"), (rel: (0, 1.2), to: "ci.end"), name: "2e")
  content("2e.content", $2e$, anchor: "south")
})
```

Directory tree

Redrawn from [file_tree.pgf](#) in Walmes M. Zeviani's *Tikz Gallery*. A tree drawn from a flat data list; elbow connectors from each parent's south to each child's west.



```
#import "@preview/cetz:0.5.2"
#set page(width: auto, height: auto, margin: 4mm)
#set text(font: ("Helvetica Neue", "Arial"), size: 10pt, fill: black)

// (depth, name) in `tree` order; an entry is a directory if the next one is deeper.
#let entries = (
  (0, "src"),
  (1, "asf"), (2, "avr32"), (3, "drivers"), (4, "canif"),
```

```

(5, "canif.h"), (5, "canif.c"),
(1, "net"), (2, "can"),
..("can.h", "can_mob.h", "can_mob.c", "can_port.h", "can_port.c", "can_trcv.h", "can_trcv.c")
.map(file => (3, file)),
(1, "..."),
)

#cetz.canvas({
  import cetz.draw: *
  set-style(stroke: (paint: black, thickness: .6pt))

  let y = 0
  let prev-depth = 0
  for (i, (depth, name)) in entries.enumerate() {
    let dir = entries.at(i + 1, default: (0,)).at(0) > depth
    y -= if depth == prev-depth { .4 } else { .6 }
    prev-depth = depth

    // Reused names: "d2" is the latest depth-2 node, i.e. the parent of each new depth-3 node.
    let node = "d" + str(depth)
    content((depth * .85, y), name: node, anchor: "west", box(
      raw(name), inset: 2.5pt, radius: 2pt,
      fill: if depth == 0 { eastern.lighten(50%) } else if dir { eastern.lighten(85%) },
      stroke: if dir { blue + .8pt },
    ))
    if depth > 0 {
      let parent = "d" + str(depth - 1) + ".south"
      line(parent, (parent, "|-", node + ".west"), node + ".west")
    }
  }
})

```